

0090 — A drop that completes a pop proves the tree still holds what the apply restored

Status: Accepted — implemented and tested **Date:** 2026-08-26
Issue: [#514](#) **Protocol:** v8 → v9

Context

A pop is not one operation. `PopStash` was removed in #501 (ADR 0078) because a single durable row cannot distinguish "nothing ran" from "applied, entry retained", so a pop is composed on the client from three unlinked requests:

1. `POST /api/stash/apply`
2. `GET /api/conflicts`
3. `POST /api/stash/drop`

Each POST takes and releases the repository guard independently, and the GET holds no guard at all. The drop's checks proved that `stash@{N}` still named `expected_oid` — that the **stash entry** had not moved. Nothing proved the **working tree** still held the changes the apply had just restored.

So this sequence deleted a user's work and reported success:

1. Apply succeeds. Guard released. The changes are in the tree.
2. Another session — or the user in a terminal — runs `git reset -hard`. The changes are gone. **refs/stash is untouched.**
3. The drop builds its plan, sees a stash entry exactly where it expects, and proceeds.

4. The entry is deleted. The client reports **"Popped"**. The work is gone from the tree *and* from the drawer.

Recovery pins mean the content remains *recoverable*. They do not make the word "Popped" true, and this is the same family as #508: the app asserting an outcome it never established.

Why the staleness gate did not already catch this

It was the first thing checked, and the answer is more interesting than "the generation is ref-shaped" — it is not. `generation_token` folds in HEAD, every ref, `refs/stash` **and the worktree status** (`planner.rs`, the `status` field). A `reset --hard` genuinely does move it.

The gate misses it on **ordering**. `build_plan` runs *before* `coordinator::lock`. `enforce_fresh` then compares the plan's observation against a live one, so it catches drift arriving **after** plan-build. Interference arriving **before** plan-build is not drift at all — the fresh plan legitimately observes the post-reset repository, and its generation matches. The drop looks around, sees a tidy repository, and is right about everything it checked.

That is why this is a separate proof inside the guard rather than a tightening of `enforce_fresh`: the gate is not wrong, it is answering a different question.

Decision

A drop states what it is the second half of, and the server proves the matching claim inside the guard.

1. The wire says why, not just what

`POST /api/stash/drop` takes `DropStashRequest { target, context }`, where `DropContext` is `Standalone` or `CompletingPop { applied_operation }`.

An enum, not `Option<OperationId>`. `None` is exactly what a caller that has quietly stopped proving anything sends, and the pop's drop would fall back to the unchecked path with nothing red anywhere. Two named answers to a question that must be answered; absent is a 400. That is a whole-window protocol move (**v9**) for the same reason v5-v8 were: the body is `deny_unknown_fields` and the context has no default.

Not every drop follows an apply. The drawer's own Drop button restored nothing and is asked to prove nothing — which is why the shape had to express both cases rather than requiring an apply id universally. An earlier draft of this change required it everywhere and would have broken standalone Drop outright.

2. The proof is the apply's own operation record

The client sends back the **operation id** it was already given, not a fingerprint it observed. Every operation records, at its terminal transition, the generation the repository had when it finished (`operations::apply_terminal`). So the server reads a fingerprint **it stored itself**, and there is nothing for the client to observe incorrectly or forge into agreement.

The id is not taken on trust. Three checks run before the attached generation means anything: the record exists, it succeeded, and it really was an `ApplyStash` of *this* entry at *this* oid.

3. The proof travels beside the operation, never inside it

`DropProof` is a planner-level argument, not a field on `GitOperation::DropStash`. Two reasons, both load-bearing:

- **It is not part of the operation's identity.** `operation_hash` is computed from the operation and idempotency compares it. "Drop this entry at this oid" is the request; which apply

preceded it is a precondition on *running*. Folding it in would make two otherwise-identical drops hash differently.

- **The journal already holds DropStash rows.** A new required field on a persisted variant makes every existing row undecodable — the exact cost #509 (ADR 0089) had just finished making honest. No reason to spend it.

This mirrors how `recovers` already travels: context beside the operation.

4. The check runs inside the guard, and nowhere else

`proof_holds` is called after `refuse_if_git_busy` and before `validate`. Any earlier and it inspects a repository another writer is still free to change — which is the original defect wearing a new hat.

5. A refusal leaves the applied changes in the tree

Owner's decision, 2026-08-26. The alternative — unwind the apply — is more work, can itself fail, and would destroy the very changes the user is trying to keep. Refusing leaves the tree as the apply left it and the entry in the drawer: nothing is lost, and the user can retry or drop by hand. Every refusal message says what was **not** done and what to check.

6. No id, no drop

If an apply succeeds but the server named no operation for it, the client has nothing to prove the tree with. `compose_pop` halts with `AppliedNotDropped` rather than falling back to an unchecked drop. A fallback there would have reintroduced the defect on precisely the path least likely to be exercised.

Alternatives considered

Server-side pop orchestration under one guard. The honest architecture, and still blocked by ADR 0078's argument: one operation row cannot represent a composite outcome. Building composite outcomes or linked child records first is a milestone, not a fix. This change does not block it and should be superseded by it.

Take the guard before building the plan. Would close the window for every operation at once. Rejected as far larger and riskier than the defect: plan construction does real work, and holding the repository guard across it lengthens every write's exclusive section. Worth revisiting on its own terms, not as a side effect of a stash fix.

A client-observed fingerprint on the request. The card this was designed from proposed it. Rejected once the operation record was found to already hold a post-execution generation: strictly more plumbing, and it puts a value the client could get wrong on the wire.

Change only the wording — stop saying "Popped" unless both halves are proven. Honest and cheap, and it leaves a known way to lose work in the product with a nicer sentence attached.

Consequences

Good. The window that could delete a user's restored work is closed. A refusal is the failure mode, not a loss. The proof is a fingerprint the server minted, so there is nothing the client can be wrong about.

Costs, stated plainly.

- **False refusals are real and expected.** Any change between apply and drop — including the user innocently editing a file — moves the generation and refuses the drop. Safe, but it will happen, and the message has to be good enough that a user knows they lost nothing.
- **v9 refuses every v8 client.** A cached client that has not learned to send a context is refused at the version gate rather than served the old unsafe path. That is the intent, and it is a real cost.
- **Standalone drops are unprotected, by design.** They restored nothing, so there is nothing to protect; the selector/oid pair is still checked.
- **proof_holds reads the durable store inside the guard,** lengthening the exclusive section slightly for a pop's drop. Measured at one user this is noise; it is stated because it is the kind of cost that stops being noise.

Verification. An end-to-end regression drives real git: a drop that cannot prove the tree refuses *and the entry is still in* `git stash list`, while a `DropProof::Nothing` drop still succeeds — the second leg being what stops a `proof_holds` that refused everything from passing. Mutation-proved two ways: the mechanism removed (red with "Dropped stash@{0}" — the defect verbatim) and the absent-record arm weakened (red by a different route). Both restored byte-identically. 978 server tests, 733 ui-bin tests, clippy clean on native and wasm.

Signed: fable · 2026-08-26T11:55:00-04:00